AutoIA: Mathematical Theory and Certification Architecture

A Self-Contained Specification Through Version 0.21.0.dev1

August 2026

Abstract

AutoIA is a research implementation of automatic validated interval analysis for existing engineering Python/NumPy programs. The central objective is to accept a deterministic engineering program, replace selected scalar and numerical operations by validated counterparts, and return output enclosures that contain the exact real semantics of every supported execution path for every input in a user-supplied interval box. The system is not based on naive line-by-line interval arithmetic alone. It combines explicit polynomials in the original uncertain inputs, interval remainders, shared auxiliary error generators, semantic certification of linear systems, path-aware execution, monotonicity and Bernstein range analysis, outward-rounded binary64 proof kernels, and prepared repeated-query certificate reuse.

This document is a mathematical specification of the current design through AutoIA version 0.21.0.dev1. Every symbol used below is defined explicitly. The document separates *proposal calculations*, which may use ordinary floating point and may be inaccurate, from *proof calculations*, whose only purpose is to establish inclusion. It also states the major current limitations and the precise sense in which prepared-session restriction can answer a new interior input box without re-executing the engineering model.

Contents

1 Problem statement and certification target **4**
1.1 What is meant by program semantics 4

2 Notation used throughout the document **5**

3 Rigorous centering of the input box **6**

4 The representation-independent enclosure invariant **6**

5 The interval representation **7**

6 Point-coefficient polynomial plus interval remainder **7**
6.1 Multi-index monomial range on a symmetric box 8

7 Shared-generator polynomial representation **8**
7.1 Why fan-out cancellation improves 9
7.2 Closure under multiplication 9

8 Rigorous polynomial algebra **9**
8.1 Addition and subtraction . . . 9
8.2 Truncated multiplication . . . 9

9 Powers, reciprocal, and division **10**
9.1 Integer powers . . . 10
9.2 Non-integer constant real powers . . . 10
9.3 Reciprocal certificate . . . 10

10 Trusted binary64 interval kernel and outward rounding **11**
10.1 Why a single outward adjustment after a long reduction is insufficient . . . 11

11 Polynomial range analysis **11**
11.1 Termwise range . . . 11
11.2 Derivative-based monotonicity . . . 12
11.3 Bernstein enclosure . . . 12
11.4 Monotonicity-reduced Bernstein . . . 12
11.5 Portfolio principle . . . 12

12 Validated elementary functions by Taylor composition **12**
12.1 Exponential . . . 13
12.2 Logarithm . . . 13
12.3 Sine and cosine . . . 13
12.4 Error function and normal distribution functions . . . 13

13 Semantic certification of parameter-dependent linear systems **14**
13.1 General polynomial solution predictor . . . 14
13.2 Residual identity for an arbitrary predictor . . . 15
13.3 Contraction theorem . . . 15
13.4 Preconditioner portfolio . . . 16

14 Generator-aware semantic linear solve **16**

15 Path-aware execution and uncertain control flow **16**

16 Adaptive subdivision controller **17**

17 Prepared sessions and repeated-query certification **18**
17.1 Exact repeated-box reuse . . . 18
17.2 Restriction of a parent certificate to an interior subbox . . . 18
17.3 Containment check for the actual centered query domain . . . 19
17.4 Many-box execution . . . 19

18 Program-level soundness theorem **19**

19 What ordinary floating-point calculations are allowed to do **20**

20 Important limitations and non-guarantees in version 0.21.0.dev1 **20**

21 A compact end-to-end mathematical example **21**

22 Summary of the current mathematical architecture **22**

1 Problem statement and certification target

Let

$$\mathcal{P} : \mathbb{R}^m \to \mathbb{R}^r$$

denote the mathematical semantics of a supported deterministic engineering program. The symbol m denotes the number of uncertain scalar input parameters and r denotes the number of scalar outputs. The input vector is

$$\mathbf{p} = (p_1, \ldots, p_m)^T \in \mathbb{R}^m,$$

and the output vector is

$$\mathbf{y} = (y_1, \ldots, y_r)^T = \mathcal{P}(\mathbf{p}) \in \mathbb{R}^r.$$

The user supplies an interval for every uncertain input,

$$p_i \in I_i = [\underline{p}_i, \overline{p}_i], \qquad i = 1, \ldots, m,$$

where $\underline{p}_i \le \overline{p}_i$ are finite binary64 numbers interpreted as exact real interval endpoints. The complete input box is the Cartesian product

$$\mathcal{P}_{\text{in}} = I_1 \times \cdots \times I_m \subset \mathbb{R}^m.$$

The exact reachable output set is

$$\mathcal{Y}^\star = \{\mathcal{P}(\mathbf{p}) : \mathbf{p} \in \mathcal{P}_{\text{in}}\}.$$

AutoIA does not promise to compute the exact interval hull of this set. The primary contract is inclusion. For every scalar output y_j, AutoIA returns an interval

$$Y_j = [\underline{y}_j, \overline{y}_j]$$

such that

$$\boxed{y_j(\mathbf{p}) \in Y_j \quad \text{for every } \mathbf{p} \in \mathcal{P}_{\text{in}}.}$$

For a piecewise/path-aware result, the system may instead return a finite union of certified branch enclosures. The reported global lower and upper bounds are then the interval hull of that finite union.

1.1 What is meant by program semantics

For generic supported scalar arithmetic, stored binary64 constants in the user program are interpreted as exact real constants equal to their stored values. Recognized mathematical primitives are interpreted according to their documented real mathematical semantics. Examples include exact real exp, log, sin, cos, erf, normal PDF/CDF, and the exact solution of a recognized linear system $Ax = b$. The semantic linear-solve primitive therefore certifies the mathematical solution of the represented matrix family, not the floating-point rounding behavior of NumPy's implementation of `linalg.solve`.

If the user manually codes an iterative or approximate algorithm that is not a recognized semantic primitive, the default goal is to certify the supported program semantics as written. AutoIA does not silently replace an arbitrary legacy algorithm by a different mathematical problem merely because the latter looks like the likely engineering intent.

2 Notation used throughout the document

This section defines the notation that appears later.

Symbol	Definition
m	Number of uncertain scalar physical inputs.
r	Number of scalar outputs.
$\mathbf{p}$	Physical input vector $(p_1, \ldots, p_m)^T$.
$\mathcal{P}_{\text{in}}$	User-supplied Cartesian input box.
$p_{c,i}$	Stored binary64 center chosen for input interval I_i.
q_i	Centered physical coordinate $q_i = p_i - p_{c,i}$.
h_i	Rigorous nonnegative half-width satisfying $q_i \in [-h_i, h_i]$.
$\mathbf{q}$	Centered physical vector $(q_1, \ldots, q_m)^T$.
$\mathcal{Q}$	Symmetric centered box $\prod_{i=1}^{m}[-h_i, h_i]$.
α	Multi-index $(\alpha_1, \ldots, \alpha_m)$ with nonnegative integer components.
$\lvert\alpha\rvert$	Total degree $\alpha_1 + \cdots + \alpha_m$.
$\mathbf{q}^\alpha$	Monomial $\prod_{i=1}^{m} q_i^{\alpha_i}$.
e_i	Unit multi-index whose i-th component is one and all others are zero.
$P(\mathbf{q})$	Stored point-coefficient polynomial in the original centered physical inputs.
R	Interval remainder, usually written $[\underline{R}, \overline{R}]$.
s	Number of active auxiliary generator variables.
$\boldsymbol{\varepsilon}$	Auxiliary vector $(\varepsilon_1, \ldots, \varepsilon_s)^T$ with each $\varepsilon_j \in [-1, 1]$.
G	Generator coefficient matrix. For a scalar, this reduces to a row/vector of coefficients.
Γ_E	Concretization/interpretation of a validated representation E.
$A(\mathbf{q})$	Parameter-dependent matrix in a semantic linear system.
$b(\mathbf{q})$	Parameter-dependent right-hand side in a semantic linear system.
$X(\mathbf{q})$	Stored polynomial predictor for a linear-system solution.
C	Ordinary floating-point point preconditioner used in a linear-system certificate.
B	Nonnegative matrix satisfying a rigorous componentwise defect bound.
c	Nonnegative vector satisfying a rigorous preconditioned residual bound.
ρ	Rigorous upper bound on $\lVert B\rVert_\infty$.
$\lVert B\rVert_\infty$	Maximum absolute row-sum norm $\max_i \sum_j \lvert B_{ij}\rvert$; since $B \geq 0$, this is $\max_i \sum_j B_{ij}$.
$\text{hull}(S)$	Smallest interval containing a scalar set S, or componentwise interval hull for a vector set.
$\text{down}(x)$	A binary64 lower endpoint known to be no greater than the exact real quantity x.
$\text{up}(x)$	A binary64 upper endpoint known to be no less than the exact real quantity x.

Componentwise vector/matrix inequalities such as

$$|v| \leq c$$

mean $|v_i| \le c_i$ for every component i. Similarly,

$$|M| \le B$$

means $|M_{ij}| \le B_{ij}$ for every entry.

3 Rigorous centering of the input box

Polynomial representations use centered coordinates. For each user interval

$$I_i = [\underline{p}_i, \overline{p}_i],$$

AutoIA chooses a finite binary64 proposal center $p_{c,i}$. Conceptually this is near the arithmetic midpoint, but the proof does not require the chosen center to be the exact midpoint. The trusted backend then rigorously encloses

$$I_i - p_{c,i}$$

using outward-rounded interval subtraction. If the resulting interval is $[\ell_i, u_i]$, AutoIA defines

$$h_i = \max(|\ell_i|, |u_i|).$$

Therefore

$$I_i \subseteq [p_{c,i} - h_i, p_{c,i} + h_i],$$

and hence every physical input in I_i admits the centered coordinate

$$q_i = p_i - p_{c,i} \in [-h_i, h_i].$$

The complete centered domain is

$$\mathcal{Q} = \prod_{i=1}^{m} [-h_i, h_i].$$

The symmetric centered domain can be slightly larger than the original user interval because outward rounding may enlarge a boundary by one or more ulps. This enlargement is sound but matters near function-domain boundaries. For example, a user interval whose exact lower endpoint is zero may produce a centered polynomial domain whose rigorous reconstructed lower endpoint is a tiny negative number. AutoIA must not silently clip that negative number when certifying $\sqrt{x}$ or $\log(x)$; the current implementation fails closed in such a case unless a representation/path/domain refinement proves the required real domain.

4 The representation-independent enclosure invariant

Every validated scalar value is associated with a representation object E and a concretization map

$$\Gamma_E(\mathbf{q}) \subseteq \mathbb{R}.$$

The fundamental invariant is

$$\boxed{z(\mathbf{q}) \in \Gamma_E(\mathbf{q}) \quad \text{for all } \mathbf{q} \in \mathcal{Q},}$$

where z is the exact real semantic quantity represented by E.

Suppose an exact mathematical operation is denoted

$$z = \mathrm{op}(z_1, \ldots, z_k).$$

A validated implementation $\mathrm{op}^{\#}$ is sound when

$$\boxed{\Gamma_{\mathrm{op}^{\#}(E_1,\ldots,E_k)}(\mathbf{q}) \supseteq \mathrm{op}\big(\Gamma_{E_1}(\mathbf{q}), \ldots, \Gamma_{E_k}(\mathbf{q})\big)}$$

for every $\mathbf{q} \in \mathcal{Q}$, with the right-hand side interpreted as the set of all values of the exact operation on arguments chosen from the input sets.

Representation conversion is similarly required to be inclusion-preserving. If E_A is converted to E_B, then

$$\Gamma_{E_A}(\mathbf{q}) \subseteq \Gamma_{E_B}(\mathbf{q})$$

for every $\mathbf{q}$. Promotion priorities in the software influence strategy only; they are not themselves part of the proof.

5 The interval representation

The simplest scalar representation is a single interval

$$E = I = [\underline{z}, \overline{z}].$$

Its concretization is independent of $\mathbf{q}$:

$$\Gamma_E(\mathbf{q}) = I.$$

This representation forgets explicit dependence on the original physical inputs, but it is robust and useful as a fallback or for calculations where interval semantics are already natural.

The natural interval extension of an exact operation is used when supported. For example, if $X = [\underline{x}, \overline{x}]$ and $Y = [\underline{y}, \overline{y}]$, then interval addition returns an outward rounded enclosure of

$$[\underline{x} + \underline{y}, \overline{x} + \overline{y}].$$

Interval multiplication encloses all four endpoint products. Division requires a denominator interval separated from zero.

6 Point-coefficient polynomial plus interval remainder

The principal dependency-preserving representation is

$$\boxed{z(\mathbf{q}) \in P(\mathbf{q}) + R,}$$

where

$$P(\mathbf{q}) = \sum_{\alpha \in \mathcal{A}} c_\alpha \mathbf{q}^\alpha$$

is a sparse polynomial with stored binary64 point coefficients c_α, $\mathcal{A}$ is a finite set of multi-indices, and

$$R = [\underline{R}, \overline{R}]$$

is an interval remainder. The concretization is

$$\Gamma_E(\mathbf{q}) = P(\mathbf{q}) + R = \{P(\mathbf{q}) + r : r \in R\}.$$

The stored coefficient c_α is interpreted as an exact real number equal to its binary64 value. When a coefficient is produced by floating-point proposal arithmetic, AutoIA separately computes a rigorous interval containing the exact coefficient that the mathematical operation would produce, subtracts the stored point coefficient, and ranges that coefficient error over the input domain into R. This separation allows fast ordinary floating-point proposals while keeping the final enclosure rigorous.

6.1 Multi-index monomial range on a symmetric box

For

$$\alpha = (\alpha_1, \ldots, \alpha_m),$$

define

$$H_\alpha = \prod_{i=1}^{m} h_i^{\alpha_i}.$$

The trusted implementation computes an upward-rounded upper bound for this nonnegative product. On the symmetric box $\mathcal{Q}$, the monomial range is

$$\mathbf{q}^\alpha \in \begin{cases} [0, H_\alpha], & \text{if every } \alpha_i \text{ is even,} \\ [-H_\alpha, H_\alpha], & \text{otherwise.} \end{cases}$$

This parity-aware rule is sharper than blindly using $[-H_\alpha, H_\alpha]$ for every monomial.

7 Shared-generator polynomial representation

The polynomial-plus-remainder representation loses correlation when the same unstructured error is reused. AutoIA therefore also supports

$$\boxed{z(\mathbf{q}, \boldsymbol{\varepsilon}) \in P(\mathbf{q}) + \sum_{j=1}^{s} g_j \varepsilon_j + R, \qquad \varepsilon_j \in [-1, 1].}$$

For a vector of r outputs this is

$$\boxed{\mathbf{z}(\mathbf{q}, \boldsymbol{\varepsilon}) \in \mathbf{P}(\mathbf{q}) + G\boldsymbol{\varepsilon} + R,}$$

where $G \in \mathbb{R}^{r \times s}$ and R denotes a componentwise interval box.

The variables ε_j are *auxiliary* uncertainty variables. They do not represent additional physical uncertain inputs. Their purpose is to preserve provenance of approximation/certification error. Two coefficients bearing the same generator identifier refer to the same ε_j; distinct identifiers are treated as independent coordinates in $[-1, 1]^s$.

This distinction prevents an unsound shortcut. If two unrelated calculations happen to produce the same numerical error radius, AutoIA does not infer that the errors are identical. Shared provenance is created only by copying/reusing a value or by a semantic operation that rigorously carries the same generator through its output predictor.

7.1 Why fan-out cancellation improves

Suppose

$$y = P(\mathbf{q}) + g\varepsilon_1.$$

If the same represented value is used twice, then

$$y - y = P - P + g\varepsilon_1 - g\varepsilon_1 = 0.$$

A pure interval remainder would instead behave like two independent copies of the same interval and could fail to cancel.

7.2 Closure under multiplication

The representation is affine in $\boldsymbol{\varepsilon}$, so it is not algebraically closed under multiplication. If

$$x = P_x + G_x\boldsymbol{\varepsilon} + R_x, \qquad y = P_y + G_y\boldsymbol{\varepsilon} + R_y,$$

then products such as

$$q_i\varepsilon_j \quad \text{and} \quad \varepsilon_i\varepsilon_j$$

appear. The current representation keeps the constant-in-$\mathbf{q}$, first-order in-$\boldsymbol{\varepsilon}$ contribution explicitly and rigorously ranges terms outside the representation family into fresh bounded error. Soundness is therefore retained without pretending that $P(\mathbf{q}) + G\boldsymbol{\varepsilon} + R$ is closed under arbitrary nonlinear operations.

8 Rigorous polynomial algebra

8.1 Addition and subtraction

Let

$$x \in P_x + R_x, \qquad y \in P_y + R_y.$$

The exact algebraic sum is

$$x + y \in (P_x + P_y) + (R_x + R_y).$$

AutoIA computes point coefficient proposals for $P_x + P_y$, rigorously encloses coefficient-rounding errors, and adds their ranged effect to the remainder. Subtraction is addition of the negated representation.

8.2 Truncated multiplication

Let the retained maximum polynomial order be K. The exact polynomial product is

$$P_xP_y = \sum_{\gamma} C_\gamma \mathbf{q}^\gamma, \qquad C_\gamma = \sum_{\alpha+\beta=\gamma} c_\alpha d_\beta.$$

For every γ satisfying $|\gamma| \le K$, AutoIA stores a point proposal $\widehat{C}_\gamma$. It rigorously encloses the exact coefficient C_γ and returns the difference

$$C_\gamma - \widehat{C}_\gamma$$

as a coefficient-error interval. That error is multiplied by a rigorous monomial range and accumulated into the remainder.

For discarded terms with $|\gamma| > K$, the full exact coefficient interval is ranged over $\mathcal{Q}$ and added to the remainder. The user-selected order K therefore changes sharpness and cost but not the inclusion contract.

The complete product also contains interactions with existing remainders. These are enclosed by ordinary interval multiplication using rigorous ranges of the retained polynomial parts.

9 Powers, reciprocal, and division

9.1 Integer powers

For $n \in \mathbb{Z}$, positive powers are constructed by validated multiplication. For $n < 0$,

$$x^n = \frac{1}{x^{-n}},$$

so AutoIA must prove that zero is excluded from the denominator range. Negative bases are valid for integer exponents.

9.2 Non-integer constant real powers

For a finite constant binary64 exponent $p \in \mathbb{R} \setminus \mathbb{Z}$, the current real branch requires the certified base range to be strictly positive. AutoIA may decompose

$$p = n + r,$$

where $n \in \mathbb{Z}$ and $|r| \leq 1/2$, then use integer powers together with $\sqrt{\cdot}$ for half powers or the identity

$$x^r = \exp(r \log x)$$

for the remaining positive-real case. An uncertain exponent is outside the current supported contract.

9.3 Reciprocal certificate

Suppose a represented denominator y has a rigorous range

$$Y = [\underline{Y}, \overline{Y}]$$

with

$$0 \notin Y.$$

Let Q be any stored polynomial proposal for $1/y$. The exact identity

$$\frac{1}{y} - Q = \frac{1 - yQ}{y}$$

allows AutoIA to bound the residual $1 - yQ$, divide by the denominator interval Y, and obtain a certified reciprocal. The quality of Q affects only the remainder width.

10 Trusted binary64 interval kernel and outward rounding

Machine rigor requires every proof-critical floating-point operation to enclose its exact real counterpart. AutoIA therefore distinguishes two classes of calculation.

1. **Proposal calculations.** These may use ordinary round-to-nearest binary64, NumPy linear algebra, heuristic scores, or approximate inverses. Their outputs are never trusted merely because they were computed.
2. **Proof calculations.** These establish interval inclusion and are performed by the audited trusted backend or by operations reducible to trusted interval primitives.

   For a scalar operation $z = x \circ y$, the trusted backend returns binary64 endpoints

$$[\underline{z}, \overline{z}]$$

known to contain the exact real value. On the reference backend this is done by round-to-nearest arithmetic followed by `nextafter` toward $-\infty$ for the lower endpoint and $+\infty$ for the upper endpoint.

The backend construction audits the binary64 environment. The audit checks, among other things, the binary64 format, NumPy `float64` size, round-to-nearest behavior, adjacent `nextafter` values, gradual underflow/subnormals, and required vectorized arithmetic behavior. If the environment does not satisfy the assumptions, AutoIA fails closed.

10.1 Why a single outward adjustment after a long reduction is insufficient

A long dot product

$$s = \sum_{i=1}^{n} a_i b_i$$

contains rounding error in every multiplication and every addition. Merely computing an ordinary BLAS result and moving it outward by one ulp does not in general enclose the exact real dot product.

The Step-9/9b trusted fast path instead encloses every product and combines those product intervals through a pairwise interval-addition tree. If each leaf contains its exact product and each internal node contains the exact sum of its two child sets, induction proves that the root interval contains the exact dot product. Vectorization changes execution speed, not the mathematical proof.

The same idea is used for point-by-interval and interval-by-interval reductions, and many independent dot-product trees may be batched simultaneously across matrix rows/columns.

11 Polynomial range analysis

A validated symbolic representation is useful only if it can be converted to a scalar interval bound. AutoIA uses several sound range engines.

11.1 Termwise range

For

$$P(\mathbf{q}) = \sum_{\alpha} c_\alpha \mathbf{q}^\alpha,$$

each monomial is enclosed separately and the resulting intervals are summed. This method is inexpensive and robust, but it may lose cancellation between terms.

11.2 Derivative-based monotonicity

For coordinate q_i, AutoIA constructs the polynomial derivative

$$\frac{\partial P}{\partial q_i} = \sum_{\alpha:\alpha_i>0} \alpha_i c_\alpha \mathbf{q}^{\alpha-e_i}.$$

If a rigorous range proves

$$\frac{\partial P}{\partial q_i} \geq 0$$

throughout the box, then P is nondecreasing in q_i; extrema occur at the appropriate endpoint of that coordinate. If the derivative is rigorously nonpositive, the opposite endpoint is used. Monotone directions can therefore be fixed before ranging the unresolved directions.

11.3 Bernstein enclosure

After mapping the active box to a unit box, the polynomial can be expressed in a multivariate Bernstein basis. The range of a polynomial over the unit box is contained in the interval hull of its Bernstein coefficients. AutoIA computes an outward-rounded version of this transformation. Because the number of Bernstein coefficients grows as a product of per-variable degrees, the method is used selectively.

11.4 Monotonicity-reduced Bernstein

If some coordinates are already proved monotone, those coordinates are fixed at appropriate endpoints and Bernstein conversion is applied only to unresolved directions. This can greatly reduce coefficient count.

11.5 Portfolio principle

If two independent rigorous methods produce intervals I_1 and I_2 that both contain the same quantity, their intersection also contains that quantity, provided the intersection is nonempty. AutoIA therefore uses range-method portfolios to improve sharpness without making correctness depend on which method happens to be tighter.

12 Validated elementary functions by Taylor composition

For a scalar represented argument x, AutoIA first obtains a rigorous argument interval

$$X = [\underline{x}, \overline{x}].$$

Choose a stored binary64 expansion center $c \in X$ and write

$$\delta = x - c.$$

For a smooth function f, a degree-K Taylor representation is

$$f(x) = \sum_{n=0}^{K} a_n \delta^n + R_{K+1}, \qquad a_n = \frac{f^{(n)}(c)}{n!}.$$

The coefficient a_n is itself represented by a rigorous interval. AutoIA chooses a point proposal $\widehat{a}_n$, stores it in the polynomial, and propagates the difference between the rigorous coefficient interval and $\widehat{a}_n$ as certified error. A rigorous analytic bound on the omitted Taylor tail R_{K+1} is then added.

This generic composition mechanism is used for exp, log, $\sqrt{\cdot}$, sin, cos, erf, the standard normal PDF, and the standard normal CDF, with function-specific coefficient recurrences and tail bounds.

12.1 Exponential

For sufficiently small $|r|$,

$$e^r = \sum_{k=0}^{N} \frac{r^k}{k!} + R_N.$$

If $|r| \le a < 1$, one convenient rigorous tail bound is

$$|R_N| \le \frac{a^{N+1}}{(N+1)!(1-a)}.$$

The trusted scalar kernel repeatedly halves a rigorous argument interval until it is small enough for the series, then reconstructs the result by trusted squaring.

12.2 Logarithm

For $x > 0$, binary exponent extraction produces

$$x = 2^k m,$$

with a scaled mantissa m. Then

$$\log x = k \log 2 + \log m.$$

Using

$$z = \frac{m-1}{m+1},$$

we have

$$\log m = 2 \operatorname{atanh}(z),$$

which is evaluated by a rigorously bounded odd series. The constant $\log 2$ is itself generated rigorously rather than trusted from a platform library.

12.3 Sine and cosine

A rigorous interval for π is generated from Machin's formula. An ordinary floating-point integer reduction index may be proposed, but the proof forms the reduced argument with the rigorous π interval. Taylor sine/cosine bounds are used when the reduced interval is small enough; otherwise a safe range such as $[-1, 1]$ is available.

12.4 Error function and normal distribution functions

The error function is

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

Moderate positive arguments use a bounded alternating Maclaurin series. Larger arguments use the complementary error function and an integration-by-parts asymptotic bracket. Odd symmetry handles negative arguments.

The standard normal density and CDF are

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2},$$

$$\Phi(x) = \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right).$$

Their polynomial Taylor coefficients are generated using differential recurrences, including

$$\phi'(x) + x\phi(x) = 0$$

and

$$\Phi'(x) = \phi(x).$$

A normal distribution with constant mean μ and standard deviation $\sigma > 0$ is handled using

$$z = \frac{x - \mu}{\sigma}, \qquad f_X(x) = \frac{\phi(z)}{\sigma}, \qquad F_X(x) = \Phi(z).$$

13 Semantic certification of parameter-dependent linear systems

Consider a represented family of linear systems

$$A(\mathbf{q})x(\mathbf{q}) = b(\mathbf{q}),$$

where $A(\mathbf{q}) \in \mathbb{R}^{n\times n}$, $x(\mathbf{q}) \in \mathbb{R}^n$, and $b(\mathbf{q}) \in \mathbb{R}^n$. The matrix and right-hand side may have retained polynomial coefficients of arbitrary supported order plus rigorous remainders.

13.1 General polynomial solution predictor

Write

$$A(\mathbf{q}) \approx \sum_{\beta} A_\beta \mathbf{q}^\beta, \qquad b(\mathbf{q}) \approx \sum_{\beta} b_\beta \mathbf{q}^\beta,$$

and propose

$$X(\mathbf{q}) = \sum_{|\alpha|\le K} x_\alpha \mathbf{q}^\alpha.$$

Formal coefficient matching gives the recurrence

$$\boxed{A_0 x_\alpha = b_\alpha - \sum_{\substack{\beta\neq 0 \\ \beta\le\alpha}} A_\beta x_{\alpha-\beta}.}$$

Here $\beta \le \alpha$ means $\beta_i \le \alpha_i$ for every coordinate. The recurrence is only a predictor generator. AutoIA may use an ordinary floating-point solve, a different reference matrix, a family-hull midpoint, or another finite proposal.

13.2 Residual identity for an arbitrary predictor

Let X be whatever predictor is actually stored. Define the true solution error

$$e = x - X.$$

Define the residual

$$r = b - AX.$$

Because $Ax = b$,

$$A(X + e) = b,$$

so

$$\boxed{Ae = r.}$$

This identity is exact and does not depend on how X was obtained.

Choose any finite point matrix C as a preconditioner. Then

$$CAe = Cr.$$

Add and subtract e:

$$e - (I - CA)e = Cr,$$

and hence

$$\boxed{e = Cr + (I - CA)e.}$$

Suppose trusted calculations establish a nonnegative vector c and nonnegative matrix B satisfying, for every admissible input,

$$|Cr| \le c,$$
$$|I - CA| \le B.$$

Taking componentwise absolute values gives

$$|e| \le c + B|e|.$$

13.3 Contraction theorem

Define

$$\rho = \|B\|_\infty = \max_i \sum_j B_{ij}.$$

If a rigorous upper bound proves

$$\boxed{\rho < 1,}$$

then the family is nonsingular throughout the represented domain and

$$\boxed{|e| \le (I - B)^{-1} c.}$$

A simpler scalar consequence is obtained by defining

$$\eta = \|c\|_\infty, \qquad \delta = 1 - \rho > 0.$$

Then

$$\|e\|_\infty \le \frac{\eta}{\delta}.$$

The implementation computes an outward-rounded upper bound for this ratio and may subsequently refine the componentwise correction by the monotone iteration

$$d^{(k+1)} = \min\left(d^{(k)},\ \mathrm{up}(c + Bd^{(k)})\right),$$

starting from a safe scalar vector bound.

13.4 Preconditioner portfolio

The center matrix inverse is a useful proposal but can be a poor contraction preconditioner. AutoIA also constructs a rigorous interval hull of the matrix family, chooses a finite representative such as its componentwise midpoint, and proposes the inverse of that representative. Every candidate is checked against the complete matrix family. Therefore a midpoint that is not physically realizable still yields a sound certificate if the final defect bound passes.

14 Generator-aware semantic linear solve

Suppose the solve inputs contain shared auxiliary generators:

$$A(\mathbf{q}, \boldsymbol{\varepsilon}) \in \widehat{A}(\mathbf{q}) + \sum_{j=1}^{s} A_j \varepsilon_j + R_A,$$

$$b(\mathbf{q}, \boldsymbol{\varepsilon}) \in \widehat{b}(\mathbf{q}) + \sum_{j=1}^{s} b_j \varepsilon_j + R_b.$$

AutoIA proposes a solution of the form

$$\boxed{X(\mathbf{q}, \boldsymbol{\varepsilon}) = P(\mathbf{q}) + \sum_{j=1}^{s} u_j \varepsilon_j.}$$

For generator ε_j, a useful sensitivity proposal is obtained from a reference matrix A_{ref} by

$$A_{\mathrm{ref}} u_j = b_j - A_j x_0,$$

where $x_0 = P(0)$. Again, this is only a proposal.

The proof then evaluates the actual residual

$$r(\mathbf{q}, \boldsymbol{\varepsilon}) = b(\mathbf{q}, \boldsymbol{\varepsilon}) - A(\mathbf{q}, \boldsymbol{\varepsilon}) X(\mathbf{q}, \boldsymbol{\varepsilon})$$

over the joint domain

$$\mathbf{q} \in \mathcal{Q}, \qquad \boldsymbol{\varepsilon} \in [-1, 1]^s,$$

and uses the same fixed-point theorem. Incoming generator identifiers are preserved in the explicit predictor when justified. The remaining componentwise correction proven by the contraction theorem is assigned fresh independent auxiliary generators because the theorem proves only a box for the residual error, not a stronger coupled geometry.

15 Path-aware execution and uncertain control flow

Consider a scalar predicate such as

$$\texttt{lhs < rhs}.$$

AutoIA forms the validated difference

$$d = \texttt{lhs} - \texttt{rhs}$$

and computes a rigorous range

$$d \in [\underline{d}, \overline{d}].$$

For the strict less-than relation:

- if $\overline{d} < 0$, the predicate is proved true throughout the current box;
- if $\underline{d} \geq 0$, it is proved false throughout the current box;
- otherwise the predicate is unresolved.

Analogous exact-real rules are used for $\leq, >, \geq, ==, != $ and scalar truth checking.

If the predicate is unresolved, AutoIA may split the physical input box and re-execute. For an exact affine one-variable predicate

$$d(p_i) = a + bp_i, \qquad b \neq 0,$$

the exact root

$$p_i^\star = -a/b$$

can be used as a direct partition point when it is rigorously recognized.

For a general nonlinear or multivariate decision surface, finitely many Cartesian boxes may not align with the exact path boundary. If subdivision cannot resolve the predicate under the allowed budget, AutoIA conservatively enumerates both feasible Boolean outcomes and certifies the continuation of each path separately. If the two branch enclosures are Y_T and Y_F, then

$$\mathcal{P}(\mathcal{P}_{\text{in}}) \subseteq Y_T \cup Y_F.$$

The branch formula may be evaluated on a superset of its true feasible domain; this can make the result wider or cause a boundary singularity to fail certification, but it cannot omit a true output.

Path-local continuation is essential. AutoIA does not immediately hull two branch values and then continue the rest of the program. Instead every feasible path carries its own validated values through downstream arithmetic, elementary functions, and semantic solves. The union is formed only at the final piecewise result.

16 Adaptive subdivision controller

Subdivision is a sharpness/resource strategy, not a correctness prerequisite. Every retained child box is independently certified. If a controller chooses a poor direction, the result may be slower or wider but does not become unsound.

The practical split score introduced in version 0.19.0.dev1 uses the already certified parent polynomial. For candidate coordinate q_d, a half-box child has a shifted center

$$s_d = \pm h_d/2$$

and new coordinate r_d satisfying

$$q_d = r_d + s_d, \qquad r_d \in [-h_d/2, h_d/2].$$

The binomial identity

$$q_d^{\alpha_d} = \sum_{b=0}^{\alpha_d} \binom{\alpha_d}{b} s_d^{\alpha_d - b} r_d^b$$

permits a cheap predicted re-centering without executing the engineering model. The score estimates derivative-sign resolution, reduced-Bernstein burden, and output-width gain. Only the selected child pair is actually certified. The controller then checks realized structural/width improvement before accepting the split.

The old expensive one-step lookahead remains available as a research oracle. Because controller decisions are outside the trusted proof kernel, agreement with that oracle is an efficiency/sharpness validation issue rather than a soundness condition.

17 Prepared sessions and repeated-query certification

Version 0.21.0.dev1 separates static program preparation from repeated box certification. Let one fixed engineering program $\mathcal{P}$ and one fixed AutoIA configuration be used for a sequence of boxes

$$\mathcal{P}_{\text{in}}^{(1)}, \mathcal{P}_{\text{in}}^{(2)}, \dots.$$

A prepared session reuses the local semantic-dispatch clone and one audited trusted backend. More importantly, it stores previously certified boxes and results in a bounded certificate cache.

17.1 Exact repeated-box reuse

If a new query box has exactly the same stored binary64 endpoints as a cached box, the previous certificate already proves the same statement. No engineering model execution is required.

17.2 Restriction of a parent certificate to an interior subbox

Suppose a parent box D has already been certified and a new query box Q satisfies

$$Q \subseteq D.$$

Set-theoretically, every certificate valid on D is automatically valid on Q. For a polynomial certificate, AutoIA can do better than returning the same parent output interval: it re-centers and re-ranges the stored polynomial on Q.

Let the old centered coordinate be

$$\mathbf{q} = \mathbf{p} - \mathbf{c}_{\text{old}},$$

and the new centered coordinate be

$$\mathbf{r} = \mathbf{p} - \mathbf{c}_{\text{new}}.$$

Then exactly

$$\mathbf{q} = \mathbf{r} + \mathbf{s},$$

where

$$\mathbf{s} = \mathbf{c}_{\text{new}} - \mathbf{c}_{\text{old}}.$$

For a monomial

$$\mathbf{q}^{\alpha} = \prod_{i=1}^{m} (r_i + s_i)^{\alpha_i}.$$

Using the multivariate binomial expansion,

$$\mathbf{q}^{\alpha} = \sum_{\beta \le \alpha} \left[\prod_{i=1}^{m} \binom{\alpha_i}{\beta_i} s_i^{\alpha_i - \beta_i} \right] \mathbf{r}^{\beta}.$$

Therefore if

$$P(\mathbf{q}) = \sum_{\alpha} c_{\alpha} \mathbf{q}^{\alpha},$$

the exact re-centered coefficient of $\mathbf{r}^{\beta}$ is

$$c'_{\beta} = \sum_{\alpha \ge \beta} c_{\alpha} \prod_{i=1}^{m} \binom{\alpha_i}{\beta_i} s_i^{\alpha_i - \beta_i}.$$

The exact shift s_i is not assumed to equal the ordinary floating-point subtraction of the two stored centers. The trusted backend forms an interval S_i containing the exact real difference. Every transformed coefficient is computed as a rigorous interval using S_i, while an ordinary binary64 point coefficient is stored as a proposal. The difference between the exact coefficient interval and the stored proposal is rigorously ranged over the new centered box and accumulated into an extra remainder.

Hence the re-centered value remains a valid certificate on the new domain. Shared-generator coefficients are copied unchanged because the coordinate shift acts on the physical polynomial P, not on the auxiliary variables $\boldsymbol{\varepsilon}$.

17.3 Containment check for the actual centered query domain

The user query Q is not quite the domain used by the polynomial executor. The executor constructs an outward symmetric centered envelope around each new binary64 center. Let that actual represented domain be $\widetilde{Q}$. Certificate restriction is used only when AutoIA proves

$$\widetilde{Q} \subseteq D.$$

This prevents the known one-ulp symmetric-centering enlargement from causing a parent certificate to be applied outside the domain on which it was proved. If this containment check fails, AutoIA performs a fresh certification instead.

17.4 Many-box execution

For a collection of N boxes, prepared execution currently supports serial, thread-chunked, and process-chunked modes. Process mode creates one prepared session and one independently audited trusted backend in every worker process. A streaming interface keeps only a bounded number of chunks in flight, so the user does not need to store all N boxes or all N full result objects in memory simultaneously.

This is an execution architecture for large repeated-query workloads. It is not yet a single dense numeric kernel with a leading N-box array dimension.

18 Program-level soundness theorem

We can summarize the architecture as a compositional theorem.

Assumptions. Assume:

1. the trusted binary64 backend's audited machine assumptions hold;
2. every proof-critical interval operation returns an enclosure of its exact real counterpart;
3. every supported validated scalar/semantic operation satisfies its enclosure contract;
4. every representation conversion is inclusion-preserving;
5. every retained path/subdivision child is certified over a domain whose union covers the relevant user input domain;
6. every semantic linear solve accepted by the system satisfies its rigorous contraction condition; and
7. the program remains within the documented supported-language semantics.

Conclusion. For every user input $\mathbf{p} \in \mathcal{P}_{\text{in}}$, the exact real output of the supported program semantics belongs to the finite union of the validated output sets returned by AutoIA. Consequently the reported global lower/upper arrays contain every exact scalar output.

Proof sketch. The proof is by induction through execution. Initial uncertain inputs are represented by enclosures containing every physical value in the input box. Every supported operation maps valid input enclosures to an output enclosure containing the exact operation on all admissible inputs. Semantic solves are validated independently by their residual/contraction theorem. If execution branches, the path engine retains every feasible outcome or subdivides the physical domain into certified children whose union covers the original domain. Thus the invariant holds on every active path. Final range methods enclose each representation's concretization, and the finite union/hull of path results contains the exact output set.

Prepared-session restriction does not alter the theorem: a parent certificate is reused only on a subdomain proved to lie inside the parent's certified domain, and the coordinate transform itself is rigorously enclosed.

19 What ordinary floating-point calculations are allowed to do

A recurring AutoIA design principle is that an approximation may be poor without making the final result unsound. Ordinary floating point is therefore acceptable for:

- polynomial coefficient proposals;
- nominal/reference linear solves;
- approximate inverses and preconditioners;
- midpoint representatives of interval matrix families;
- branch split scores and controller heuristics;
- candidate Taylor centers;
- integer argument-reduction proposals for trigonometric functions;
- cache-choice heuristics such as choosing the smallest containing parent certificate.

Such calculations may affect sharpness, performance, or whether a proof attempt succeeds. They must not be used as the final reason an inclusion is accepted. The proof always returns to rigorous residual, interval, range, or domain checks.

20 Important limitations and non-guarantees in version 0.21.0.dev1

The current system is a substantial research prototype, not a proof assistant for arbitrary Python. Important limitations include the following.

1. **Exact hulls are not promised.** Returned intervals can be wider than the exact range.
2. **Supported-language boundary.** Arbitrary Python side effects, uncertain array indices, uncertain Boolean masks, general `np.where`, and general uncertainty-dependent loop termination are not part of the frozen contract.

3. **Boundary-domain over-enclosure.** Symmetric polynomial centering can enlarge an exact physical boundary slightly. This can make a mathematically valid $\sqrt{\cdot}$ or log query fail closed near zero.

4. **Shared-generator family is affine in auxiliary variables.** Terms such as $q_i\varepsilon_j$ and $\varepsilon_i\varepsilon_j$ are bounded back into fresh error rather than retained explicitly as a richer structure.

5. **Generator compression is not yet implemented.** The number of auxiliary IDs can grow in long calculations.

6. **Piecewise certificate restriction is limited.** Exact cache reuse works for pathwise results, but sharp geometric restriction of arbitrary cached piecewise branch domains is not yet a general many-box feature.

7. **Large many-box execution is not yet a true vectorized polynomial batch kernel.** Current process parallelism and certificate reuse are useful, but unrelated boxes are still separate certifications.

8. **GPU proof execution is not implemented.** The public prepared/batch architecture is designed to permit future CPU/GPU numeric-buffer backends.

9. **External semantic primitives are limited.** Linear solve is the major structured semantic operation. General nonlinear roots, ODE/DAE solvers, eigensystems, optimizers, and arbitrary legacy algorithm recognition are future work.

21 A compact end-to-end mathematical example

Consider a two-parameter model

$$p_1 \in [0.9, 1.1], \qquad p_2 \in [-0.1, 0.1],$$

with

$$A(\mathbf{p}) = \begin{bmatrix} 2 + p_1^2 & 0.2 \\ -0.1p_2 & 1.5 + 0.05p_1 \end{bmatrix},$$

$$b(\mathbf{p}) = \begin{bmatrix} 1 + 0.1p_2 \\ 0.5 + 0.05p_1^2 \end{bmatrix},$$

and let $u(\mathbf{p})$ solve

$$A(\mathbf{p})u(\mathbf{p}) = b(\mathbf{p}).$$

Suppose the program returns

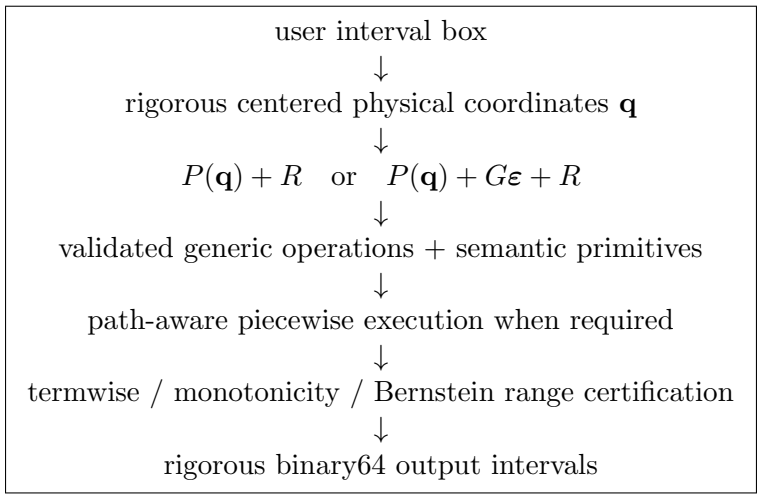
$$y_1 = u_1, \qquad y_2 = e^{u_2}, \qquad y_3 = u_1 - u_1.$$

AutoIA first constructs rigorous centered coordinates for p_1, p_2. Polynomial arithmetic retains the quadratic occurrences of p_1 up to the chosen order. The semantic solve constructs a polynomial (and, in generator mode, auxiliary sensitivity) predictor, then proves a contraction bound on the complete matrix family. The exponential is composed by the validated Taylor machinery. Shared generator provenance can preserve exact downstream cancellation in y_3, so y_3 can certify as zero even when intermediate approximations carry error.

If a prepared session first certifies the master box and later receives a smaller interior box, the stored output polynomial certificates may be re-centered using the exact binomial coordinate transform with rigorous coefficient-error capture. The model and the semantic solve need not run again if the new centered domain is proved to lie inside the parent domain.

22 Summary of the current mathematical architecture

The current architecture can be summarized by the chain



with a prepared-session layer that can reuse the static dispatch architecture and, more importantly, restrict already-proved certificates to eligible interior query boxes.

The central theoretical thesis is that sharp automatic interval analysis for engineering code is best treated as a structured validated-execution problem: retain original-input dependence explicitly, introduce auxiliary variables only when their provenance is proved, treat important numerical operations according to their mathematical semantics, and confine correctness-critical floating-point work to a small audited enclosure kernel. Performance strategies are then free to be aggressive because any proposed approximation must still pass an independent rigorous certificate before it is accepted.